

Частина 1

Віртуальне оточення: venv і Poetry

Простими словами — так, щоб зрозуміла навіть дитина.

Навіщо це потрібно? Коробки з LEGO

Уяви, що вдома в тебе багато наборів LEGO: замок, піратський корабель і космічна станція. Якщо висипати всі деталі в одну купу на підлогу, ти довго шукатимеш потрібну деталь, а деталі з різних наборів почнуть плутатися.

Тому кожен набір зберігають у своїй коробці. Відкрив коробку «Замок» — і там саме ті деталі, які потрібні для замку, і нічого зайвого.

У програмуванні так само. Деталі — це бібліотеки (чужий готовий код, який ми підключаємо до своєї програми). Коробка — це віртуальне оточення. У кожного проєкту своя коробка зі своїми бібліотеками.

- У кожного проєкту — свої бібліотеки.
- Без ізоляції версії бібліотек конфліктують між собою.
- Віртуальне оточення — окрема «мікросистема» для кожного проєкту.

Приклад конфлікту: старому проєкту потрібна бібліотека версії 1.0, новому — версії 3.0. У спільній купі може лежати лише одна версія, і один із проєктів зламається. В окремих коробках обидва живуть спокійно.

1. Створюємо віртуальне оточення (venv)

venv — це інструмент, який уже вбудований у Python. Окремо встановлювати його не треба. Команду виконуємо в терміналі, перебуваючи в папці свого проєкту.

```
# створення оточення (папка venv з'явиться в проєкті)
python -m venv venv
```

Розберемо команду по шматочках:

- python — запускаємо Python;
- -m venv — просимо його запустити вбудований модуль venv;
- venv (друге слово) — ім'я папки, куди все покласти. Це просто ім'я, можна назвати .venv — так роблять найчастіше.

Активація — «відкрити коробку»

Оточення створено, але Python поки що про нього не знає. Його треба активувати — тобто сказати терміналу: «тепер бери деталі з цієї коробки».

```
# Windows (PowerShell)
.\venv\Scripts\activate

# macOS / Linux
source venv/bin/activate
```

Після активації в консолі з'явиться префікс (venv) — це означає, що ти працюєш усередині оточення:

```
(venv) PS C:\MyProject> python --version
Python 3.11.8
```

Якщо PowerShell лається «виконання сценаріїв вимкнено», один раз виконай:
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

Щоб «закрити коробку» і повернутися у звичайний режим:

```
deactivate
```

2. Робота з пакетами (pip)

pip — це магазин деталей. Він завантажує бібліотеки із сайту pypi.org і кладе їх в активну коробку. Важливо: спочатку активуй оточення, потім став пакети.

```
pip install requests      # встановити пакет
pip uninstall requests    # видалити пакет
pip list                  # подивитися, що встановлено
```

Список деталей: requirements.txt

Коробку з деталями незручно возити із собою — вона важка (папка venv багато важить і її не зберігають у git). Замість цього записують список: що і якої версії лежало всередині.

```
# зафіксувати всі встановлені пакети у файл
pip freeze > requirements.txt
```

А на іншому комп'ютері за цим списком збирають таку саму коробку:

```
pip install -r requirements.txt
```

Правило: папку venv у git не додають, а requirements.txt — обов'язково додають. Список деталей маленький, а за ним будь-хто збере точно таке саме оточення.

3. Альтернатива: Poetry

Poetry — помічник, який робить те саме, але автоматично: сам створює віртуальне оточення і сам веде список залежностей.

```
pip install poetry      # встановити Poetry
poetry init -n          # створити pyproject.toml без запитань
poetry install          # створити .venv і поставити залежності
```

Poetry створить папку `.venv` автоматично, а список бібліотек збереже у файлі `pyproject.toml` — це сучасна заміна `requirements.txt`.

Корисні команди Poetry:

```
poetry add requests     # поставити пакет і записати в pyproject.toml
poetry remove requests  # видалити пакет
poetry run python my.py # запустити скрипт усередині оточення
poetry shell            # увійти в оточення (як activate)
```

Що обрати новачкові? Почни з `venv + pip` — так зрозуміліше, що відбувається всередині. Poetry знадобиться пізніше, на великих проєктах.

venv чи Poetry — коротко

- `venv + pip`: вбудований у Python, простий, усе робиш руками.
- Poetry: встановлюється окремо, більше робить сам, свій формат файлів.
- Сенса в обох один: у кожного проєкту — своя коробка з бібліотеками.

Шпаргалка: усі команди підряд

```
python -m venv venv          # 1. створити оточення
.\venv\Scripts\activate     # 2. активувати (Windows)
source venv/bin/activate    #   активувати (macOS/Linux)
pip install requests        # 3. поставити бібліотеку
pip list                    # 4. перевірити, що встановлено
pip freeze > requirements.txt # 5. зберегти список
pip install -r requirements.txt # відновити за списком
deactivate                  # 6. вийти з оточення
```

Чек-лист: перевір себе

- ☐ Я створив папку проєкту і відкрив у ній термінал.
- ☐ Я виконав `python -m venv venv` і побачив нову папку.
- ☐ Я активував оточення і бачу префікс (`venv`) у консолі.
- ☐ Я встановив пакет і знайшов його у виводі `pip list`.
- ☐ Я створив `requirements.txt` командою `pip freeze`.
- ☐ Я вмію виходити з оточення командою `deactivate`.

Корисні посилання

- Virtual environments (Real Python)
<https://realpython.com/python-virtual-environments-a-primer/>
- What is pip? (Real Python)
<https://realpython.com/what-is-pip/>
- Документація pip
<https://pip.pypa.io/en/stable/>
- Каталог пакетів PyPI
<https://pypi.org/>
- Документація Poetry
<https://python-poetry.org/docs/>